mystl::vector

C++20-də `std::vector`-un sıfırdan implementasiyası:
dizayn, testlər və ölçmələr

Rahim Shikhgayibov
github.com/RahimShikhgayibov/cpp-vector-from-scratch

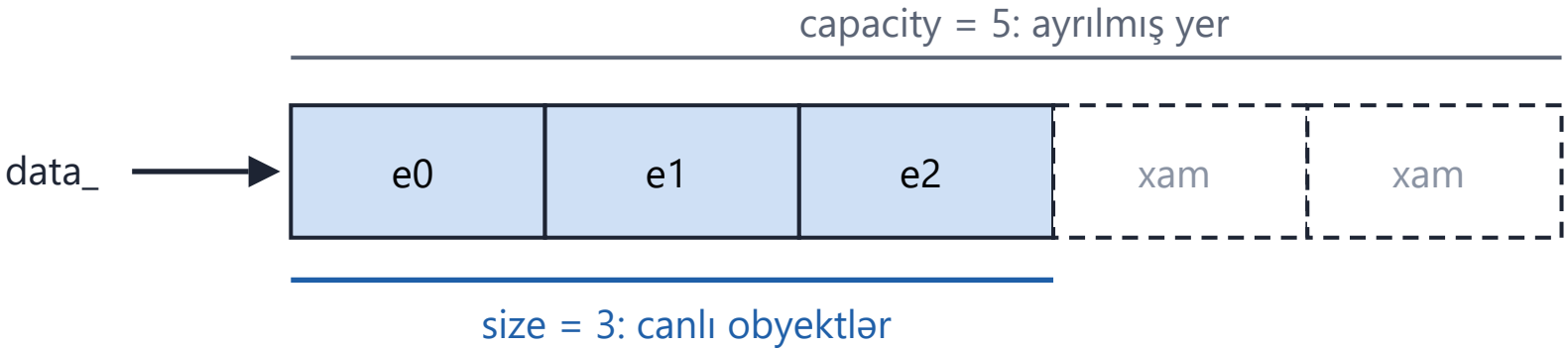
Problem və məqsəd

- **Dinamik massiv:** elementlər ardıcıl yaddaşda, O(1) indeksləmə, lazım olduqca böyüyür
- `std::vector`-u **sıfırdan** yazmaq: C++-ın ən çətin mövzuları bir layihədə
- Obyektin ömrü, xam yaddaş, rule of five, move semantics, exception safety, template
- **Əhatə:** tam interfeys, allocator və `vector<bool>` xaric

7 tədqiqat sualı

1. `push_back` amortized O(1)-dirmi?
2. Böyümə: 2× yoxsa 1.5×?
3. `reserve` nə qədər qazandırır?
4. Copy və move; `noexcept`
5. Ortaya `insert` / `erase`
6. Cache: vector və linked list
7. `std::vector`-a nə qədər yaxın?

Yaddaş modeli: [qurulmuş | xam]



- Yaddaş: `::operator new` (xam) · obyekt: `std::construct_at` (placement new) · silmə: `std::destroy_at`
- İnvariant: `size ≤ capacity`; `[0, size)` canlı, `[size, capacity)` xam; buffer-in tək sahibi

Niyə `new T[]` yox?

```
OldVector (new T[]):
  reserve(3):   ctor #2, #3, #4   ← lazımsız
  push_back:    assign #2 ← #1
  pop_back:                       ← dağıdılmır
  ~vector:      dtor #4, #3, #2

mystl::vector (xam yaddaş):
  reserve(3):                     ← heç nə
  push_back:    copy #6 ← #5
  pop_back:     dtor #6           ← dərhal
  ~vector:      —
```

- `new T[n]` yer ayıranda **n obyekt qurur**
- 1. Default constructor-u olmayan tip kompilyasiya olunmur
- 2. `reserve` lazımsız obyektlər qurur
- 3. `pop_back` obyekti dağıda bilmir
- **Həll:** yaddaşı və obyektləri ayırmaq

Təkamül: v1 → v5

Versiya	Əlavə	Həll edilən problem
v1	`int*`, size, capacity, 2× böyümə	sabit ölçülü massiv
v2	copy ctor, copy-and-swap	shallow copy → **double free** (`0xC0000374`)
v3	`template <typename T>`	yalnız `int`
v4	xam yaddaş + placement new	`new T[]`-in 3 problemi, `push_back(v[0])` bug-ı
v5	move semantics, `move_if_noexcept`	lazımsız kopyalar, `unique_ptr`
Faza B	iterator-lar, 7 ctor, `insert` / `erase` / `emplace`, `<=>`, growth policy	tam `std::vector` interfeysi

Hər versiya ayrıca commit-dir; hər addımın problemi nümayiş proqramı ilə göstərilib.

Exception safety və alias tələsi

Strong guarantee: exception olsa, vektor dəyişmir

```
1. yeni buffer ayır
2. YENİ elementi qur        ← v[0] hələ canlıdır
3. köhnələri köçür (move_if_noexcept)
   ↳ hər addımın öz catch-i
4. köhnə buffer-i at        (noexcept)
```

289 exception ssenarisi ilə yoxlanılıb: hər mümkün nöqtədə

Tapılan bug: v3-də `v.push_back(v[0])`

```
v3:  köhnə massiv silinir,
     SONRA v[0] oxunur
     → use-after-free

"zəhərli" destructor ilə test:
  v3: v[2] = -1
  v4: v[2] = 42
```

Mürəkkəblik və amortized O(1)

Əməliyyat	Zaman
`[]`, `at`, iterator-lar, `size`	O(1)
`push_back`, `emplace_back`	amortized O(1)
`insert` / `erase` (pos)	O(n − pos)
copy, `clear`, `==`	O(n)
move, `swap`	O(1)
`reserve`, `shrink_to_fit`	O(n)

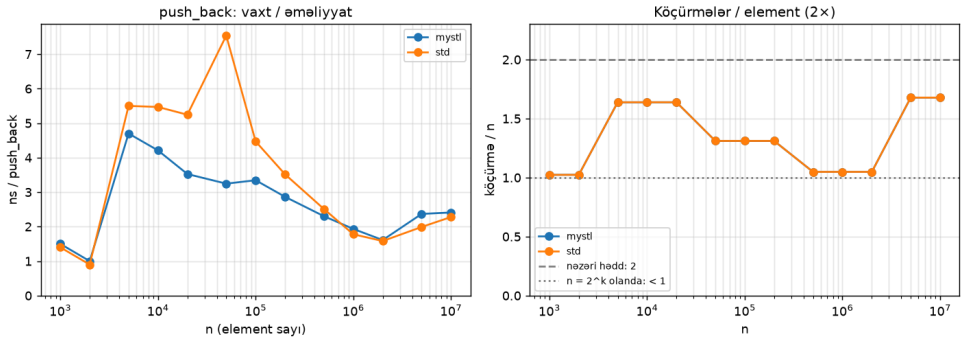
Aggregate metodu (2×):

$$T(n) = n + \Sigma\, 2^j < n + 2n = 3n$$

→ əməliyyat başına < 3 = **O(1)**

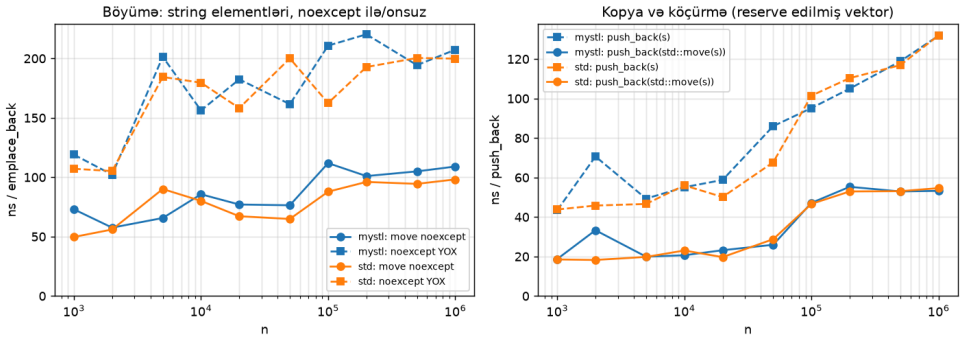
Ümumi α: köçürmələr < α·n / (α − 1) → 2×: < 2n, 1.5×: < 3n
Sabit artım (+k) olsaydı: ~$n^2/2k$ = **O(n^2)**

Sual 1: `push_back` amortized O(1)-dirmi? Bəli



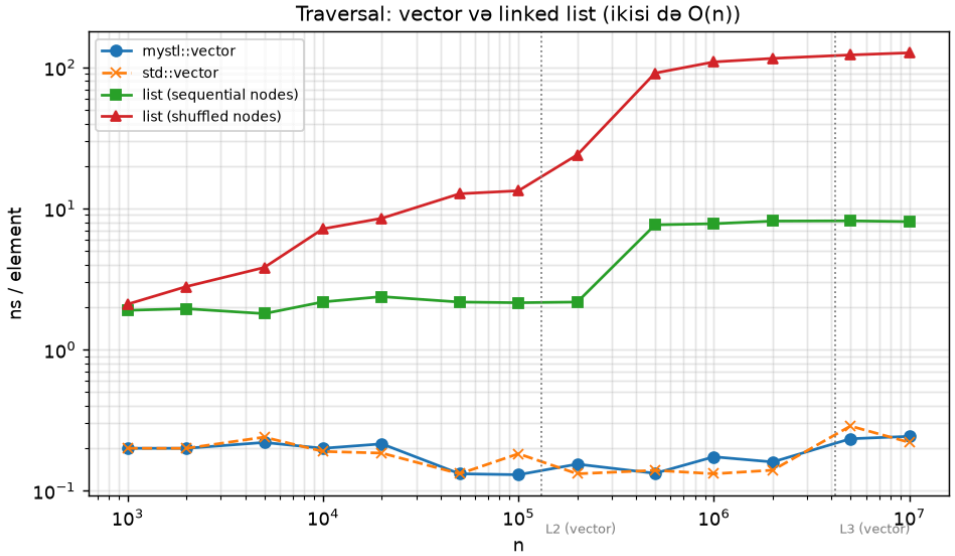
- n 10^4 dəfə artır, vaxt **1–5 ns** aralığında qalır
- Köçürmələr həmişə **< 2n**, n = 10^7: 1.68n
- Pilləli forma nəzəriyyədəndir: n-in 2^k-ya nisbəti
- `std::vector` ilə köçürmə sayı **eynidir**

Sual 4: `noexcept` unudulanda



- Fərq yalnız move ctor-dakı `noexcept` sözüdür
- Böyümədə `move_if_noexcept` **kopyalamağı** seçir
- ~2× yavaşlama (109 → 207 ns/op)
- Səssizdir: kod düzgün, sadəcə yavaş
- Kopya və move: ~2.5× fərq

Sual 6: cache — ikisi də O(n)

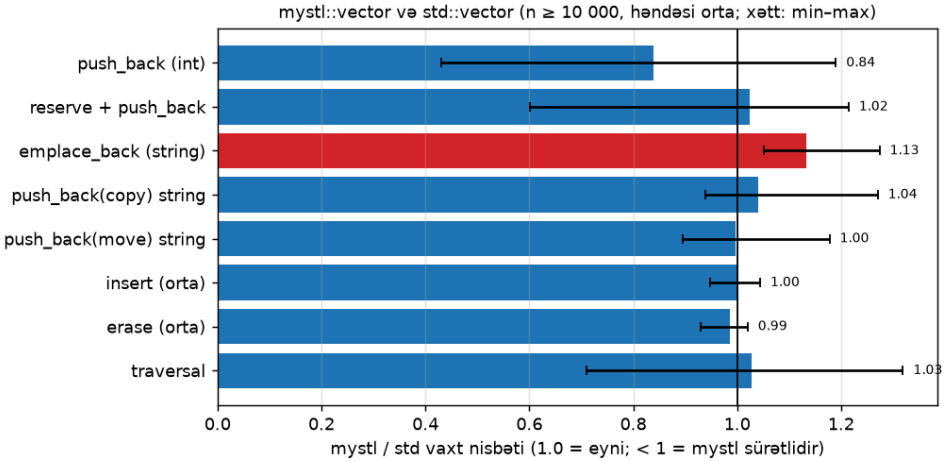


- vector: **~0.2 ns**/element (prefetch)
- list, ardıcıl düyünlər: ~2–8 ns
- list, qarışıq düyünlər: **~127 ns**
- 500–600× fərq
- L3 (16 MB) dolanda sıçrayış

Digər suallar: 2, 3, 5

Sual	Nəticə
2. 2× və 1.5×	Trade-off: 2× ~25% sürətli (25 və 41 yenidən ayırma, n = 10^7); 1.5× daha az boş yer (orta 17% və 26%, maksimum 33% və 50%)
3. `reserve`	**1.8–14×** qazanc; ən böyüyü orta ölçülərdə (10^5: 9.5×)
5. ortaya `insert` / `erase`	O(n): n = 10^7-də əvvələ bir `insert` **~3.8 ms**, ortaya ~1.6 ms, sona ~20 ns

Sual 7: `std::vector`-a yaxınlıq



- mystl / std nisbəti, n ≥ 10^4, həndəsi orta
- 8 əməliyyatdan 7-si: **0.84–1.04**
- `emplace_back(string)`: ~10% yavaş
- **Praktik olaraq eyni sürət**

Testlər

177
unit test
10 958 yoxlama

2M
differential əməliyyat
`std::vector` ilə
müqayisə

99%
sətir coverage
(gcov)

0
ASan/UBSan xətası
(Linux, g++ 14.4)

- **Sayğac tipi:** hər testin sonunda canlı obyekt = 0 (sızma və ikiqat destructor yoxdur)
- **Exception injection:** 289 ssenari, strong guarantee hər nöqtədə
- **Mutation testing:** bilərəkdən salınan 6 səhvin hamısını testlər tutdu
- **Positive control:** ASan bilərəkdən səhv proqramlarda leak və double free-ni tutur

Trade-off-lar və nəticə

Trade-off-lar

- `assign` həmişə yeni buffer ayırır → strong guarantee
- Aralıq `insert`: `std::rotate` → sadə, alias-a qarşı təhlükəsiz
- Iterator = pointer: debug yoxlaması yoxdur
- Əhatə xaricində: allocator, `vector<bool>`

Nəticə

- Tam `std::vector` interfeysi, 5 versiyalıq təkamül
- Nəzəriyyə ölçmə ilə təsdiqləndi: köçürmələr < 2n
- Big-O hər şeyi demir: cache **600×**, `noexcept` **2×**
- `std::vector` ilə eyni sürət

Suallar?